

ECE 449 Test Results



**University
of Victoria**

Formats A, B and L Demos

Authors:

Robin Zerr V01013510

Ava Birtwhistle V00970244

ECE 449 – Computer Systems and Architecture

April 6th, 2026

Note: Although Format A was demonstrated in class, the structure of our project changed as our project developed over time. Format A was retested to ensure system accuracy.

1. Introduction

This document is intended to record the results of tests conducted for ECE 499, including outcomes and supporting simulation waveform. The Following formats were tested: Format A, Format B, Format L along side the Final Tests and Factorial Tests.

2. Test Benches

A table was created to show the intended outputs compared to the actual outputs recorded for the various tests conducted.

Test	File	Test Bench	Pass / Fail	Expected Output	Actual Output
Format A Test	formata_test.mem	tb_formata_test.vhd	Pass	out_port = 0x0060	out= 0x0060
Format B Test 1	formatbtest1.mem	tb_formatbtest1.vhd	Pass	r0 = 0x0002 r1 = 0x0003 r2 = 0x0002 r3 = 0x0005 out_port = 0x0000	r0 = 0x0002 r1 = 0x0003 r2 = 0x0002 r3 = 0x0005 out_port = 0x0000
Format B Test 2	formatbtest2.mem	tb_formatbtest2.vhd	Pass	r1 = 0x009e r2 = 0x004f r6 = 0x0000 r7 = 0x0222 out_port = 0x0000	r1 = 0x009e r2 = 0x004f r6 = 0x0000 r7 = 0x0222 out_port = 0x0000
Format B Test 3	formatbtest3.mem	tb_formatbtest3.vhd	Pass	r6 = 0xffff6 (-10) ₁₀ out_port = 0x0000	r6 = 0xffff6 (-10) ₁₀ out_port = 0x0000
Format L Test 1	formatltest1.mem	tb_formatltest1.vhd	Pass	out_port = 0x00bf	out_port = 0x00bf
Format L Test 2	finaltest1.mem	tb_finaltest1.vhd	Pass	r1 = 0x050f r2 = 0x0600 r3 = 0x050f	r1 = 0x050f r2 = 0x0600 r3 = 0x050f
Format L Test 3	finaltest2.mem	tb_finaltest2.vhd	Pass	out_port = 0xffffa (-6) ₁₀	out_port = 0xffffa (-6) ₁₀

Factorial Test	factorialtest.mem	tb_factorialtest.vhd	Pass	out_port = 0x0078	out_port = 0x0078
Factorial Overflow Test	factorialtestoverflow.mem	tb_factorialtestoverflow.vhd	Pass	out_port = 0x0000 alu_result = 0x9d80 (-25216) ₁₀	out_port = 0x0000 alu_result = 0x9d80 (-25216) ₁₀

3. Simulation Waveforms

Format A Test Waveform:

Format A was run with two external inputs, loading r1 = 0x0003 and r2 = 0x0005 through back-to-back IN instructions. The datapath then performed ADD r3,r2,r1, SHL r3,2, and MUL r2,r1,r3 before OUT r2, which produced the expected final value of 0x0060 (96) on the output port. This test confirms correct input timing, ALU arithmetic, shift-immediate handling, and output-port routing for a simple straight-line program.

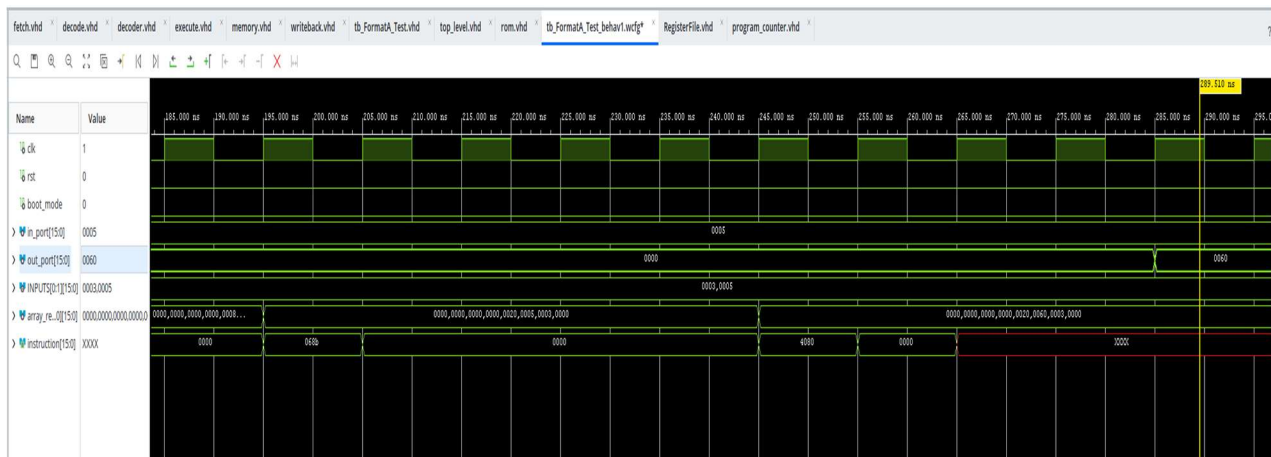
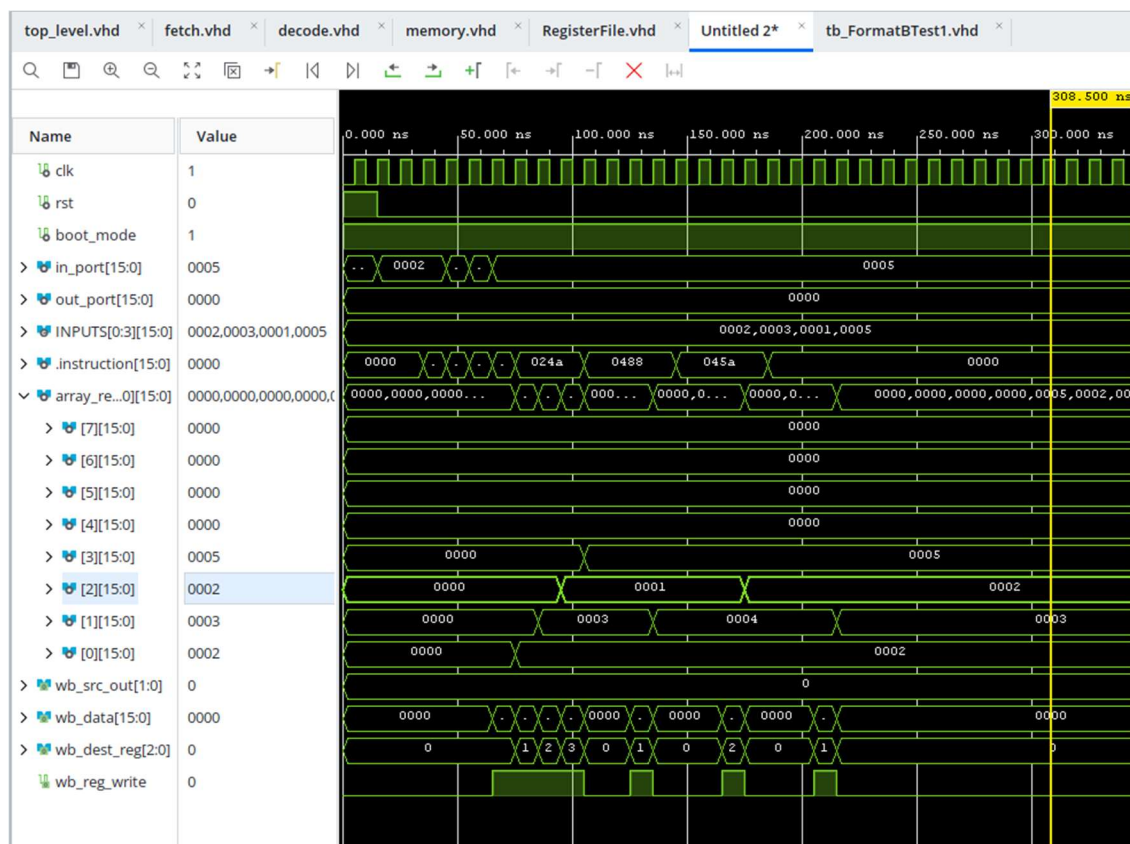
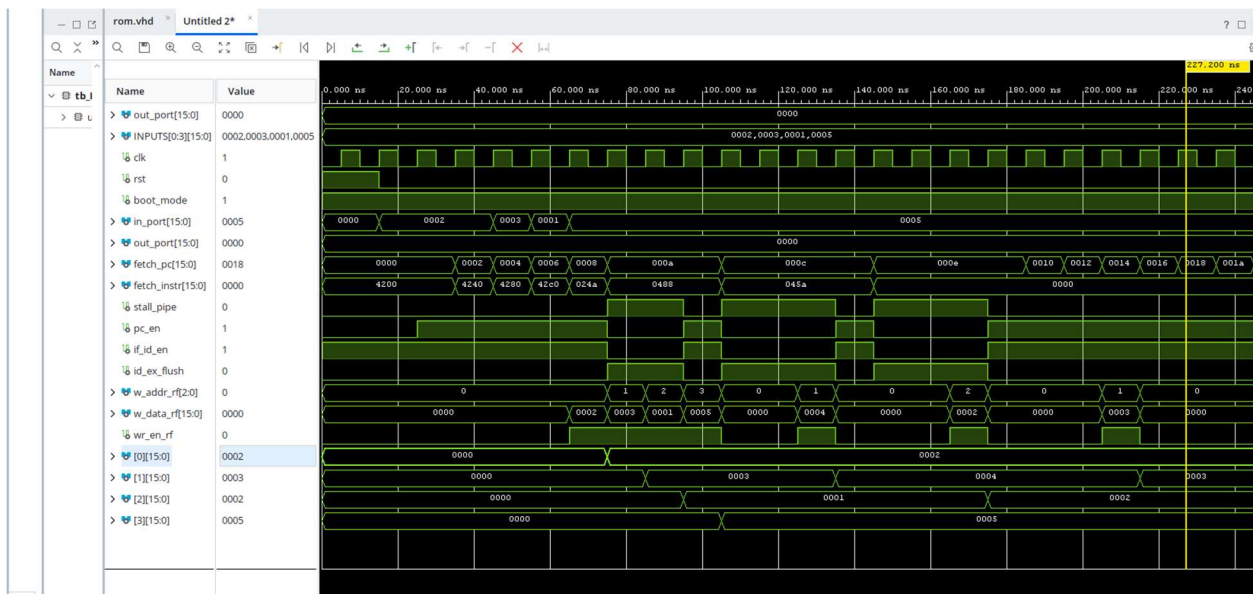


Figure 1: Format A Test Waveform

Format B Test 1 Waveform:

Format B Test 1 used four input values, r0 = 0x0002, r1 = 0x0003, r2 = 0x0001, and r3 = 0x0005, and it does not contain an OUT instruction. The intended final architectural state is r0 = 0x0002, r1 = 0x0003, r2 = 0x0002, and r3 = 0x0005 after the ADD and two dependent SUB instructions retire. This test is mainly a hazard-handling check because the later arithmetic instructions depend on recently written register values and must be protected by the stall logic.



Format B Test 2 Waveform:

Format B Test 2 validates relative branching, conditional branching, and subroutine return behavior in the pipelined CPU. The expected key final values are r1 = 0x009E (158), r2 = 0x004F (79), r6 =

0x0000, and r7 = 0x0222, with no output-port update required. This test demonstrates that branch targets are calculated correctly, the return address is preserved in r7, and loop control behaves properly across repeated branch decisions.

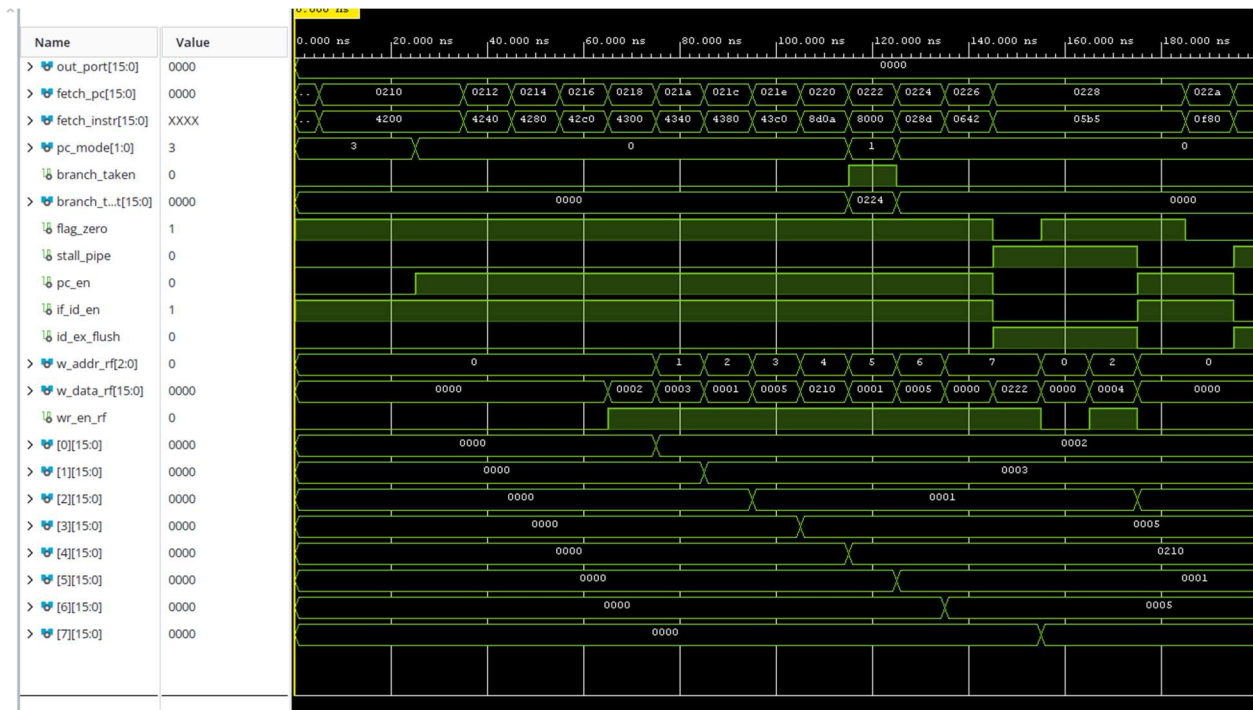


Figure 4: Format B Test 2 Waveform 0ns-200ns

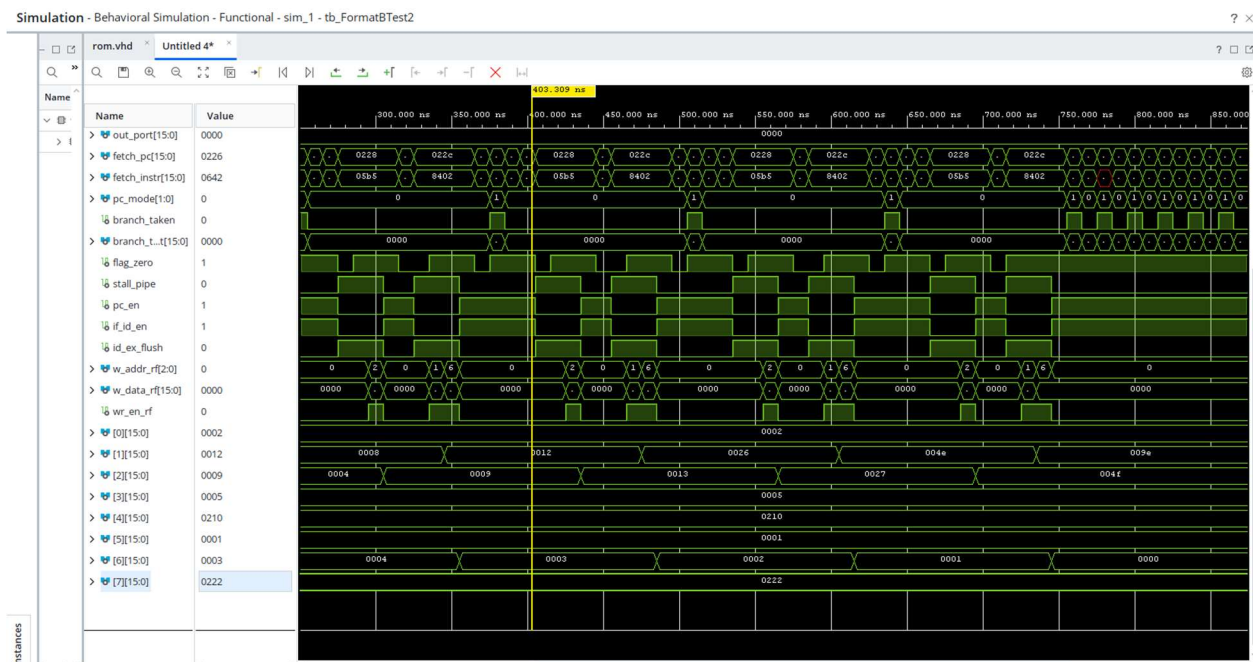


Figure 5: Format B Test 2 Waveform 250ns - ~900ns. Looping behavior evident from instructions waveform.

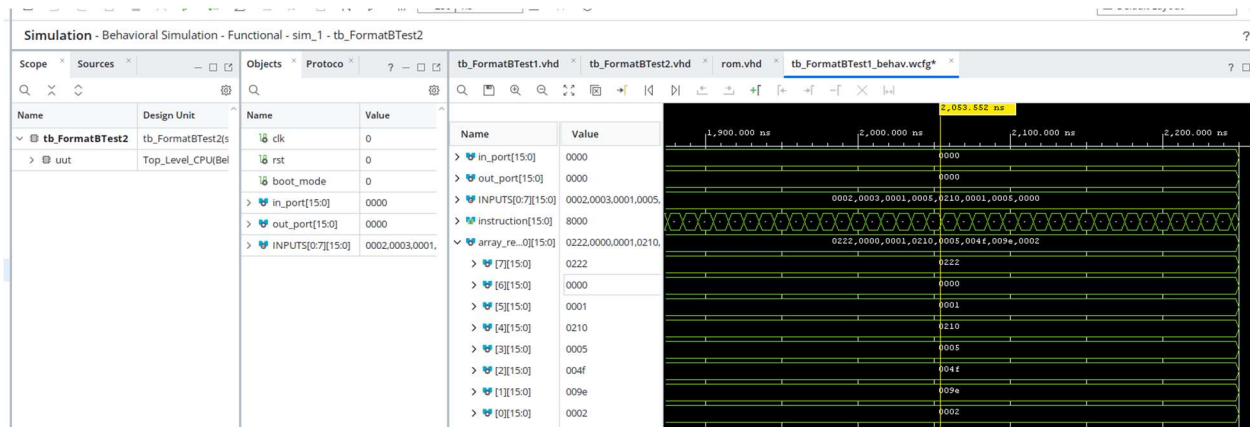


Figure 6: Format B Test 2 Waveform expected outputs

Format B Test 3 Waveform:

Format B Test 3 exercises signed arithmetic by driving $r0 = 0xFFFFE$ (-2), $r1 = 0x0003$, $r2 = 0x0001$, and $r3 = 0x0005$ through the input path. The key expected result is $r6 = 0xFFFF6$, which corresponds to -10 in signed 16-bit arithmetic, while the output port remains unchanged. This test confirms that negative values propagate correctly through subtraction and multiplication and that the register file and ALU preserve signed two's-complement behavior.

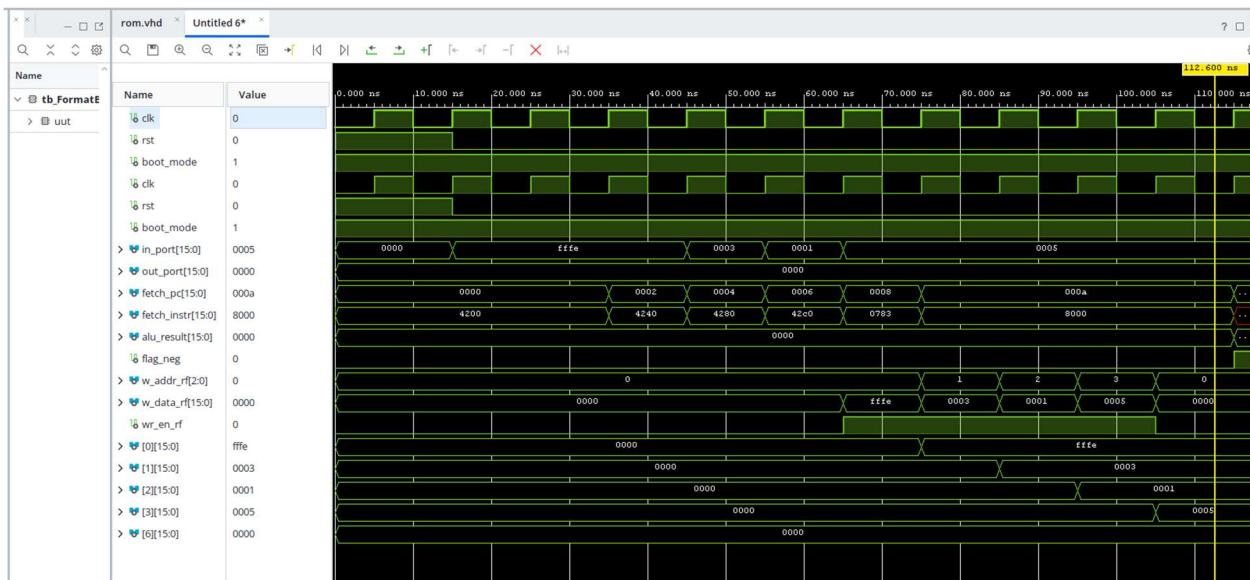
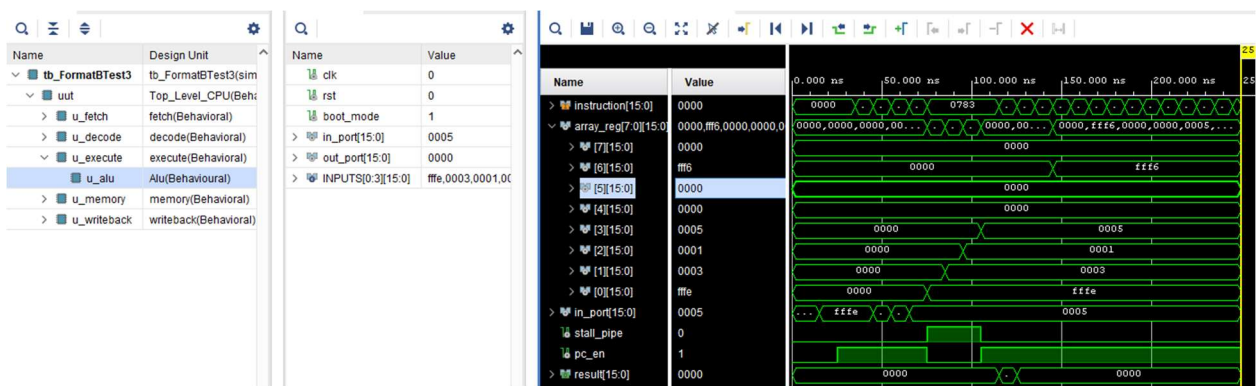
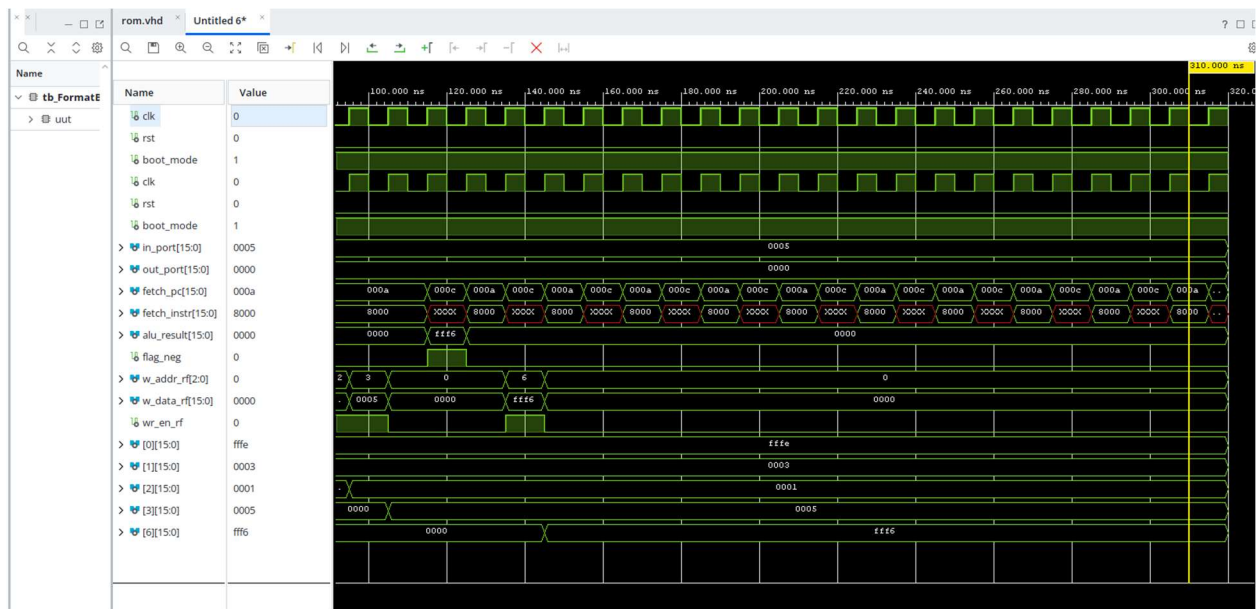
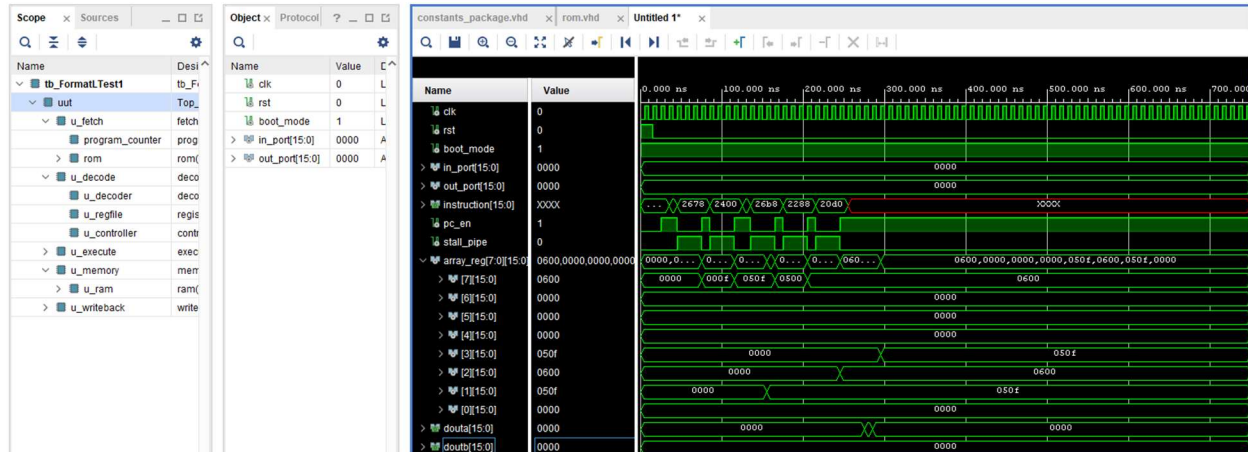


Figure 7: Format B Test 3 Waveform 0-120ns



Format L Test verifies the implemented LOADIMM, MOV, STORE, and LOAD instructions using the memory path. The intended final values are r1 = 0x050F, r2 = 0x0600, and r3 = 0x050F, showing that the value stored to memory at address 0x0600 is read back successfully. This test confirms that immediate construction in r7 works, address generation is correct, and the RAM read/write path is integrated properly with writeback.



Final Test 1 Waveform:

Final Test 1 uses an input value of 0x0005 and is expected to drive the external output port to 0x00BF, which is decimal 191. Internally, the program iterates through a counted loop and only reaches the final OUT instruction after the loop exit and branch resolution complete, so the waveform must be observed long enough for retirement. This test checks loop control, arithmetic accumulation, and the ability to hold a correct result until it is presented on the output port.

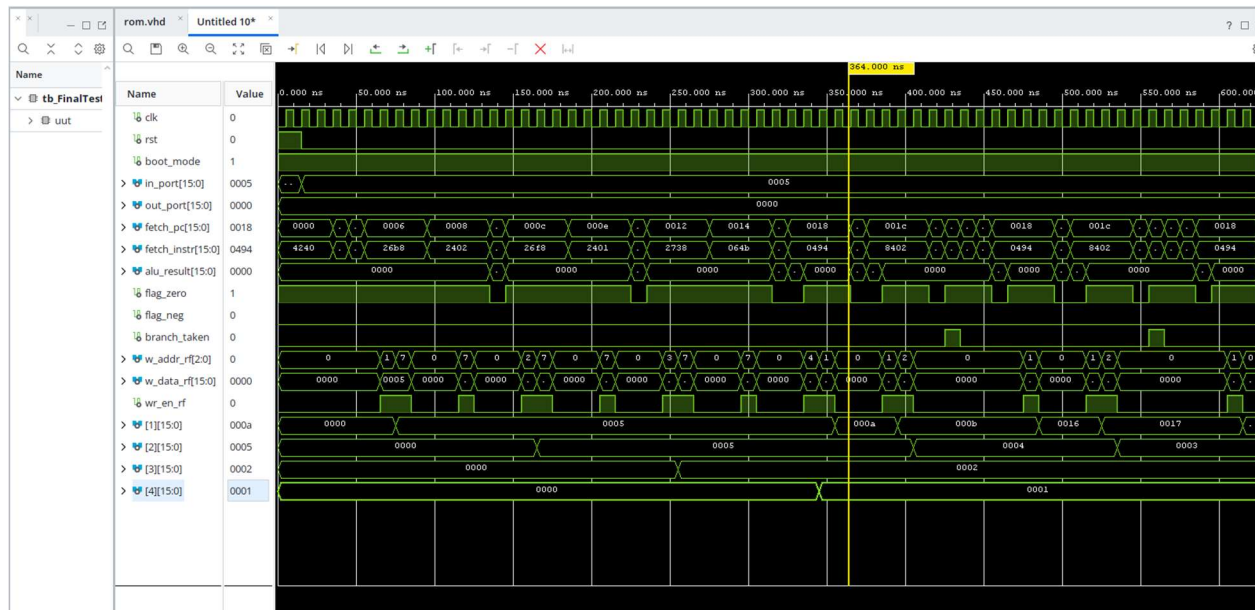


Figure 13: Final Test 1 Waveform, Loading Registers 1,2,3,4

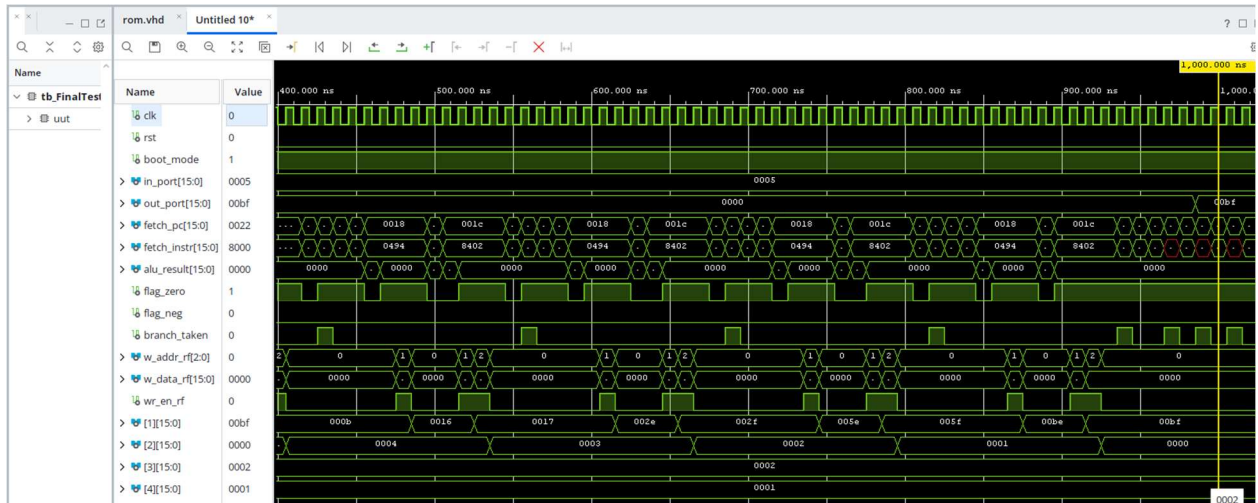


Figure 14: Final Test 1 Waveforms showing looping behavior

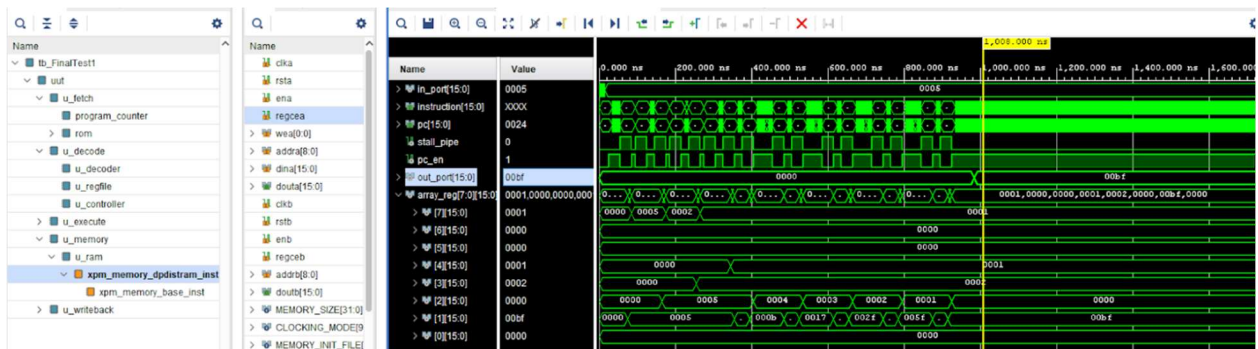


Figure 15: Final Test 1 Waveform expected outputs

Final Test 2 Waveform:

Final Test 2 also starts from an input of 0x0005 but is expected to produce 0xFFFFA on the output port, which is -6 in signed 16-bit arithmetic. Like Final Test 1, it relies on iterative control flow and therefore requires enough simulation time for the final branch exit and OUT instruction to retire. This test confirms that the same branch and arithmetic infrastructure can support a different final signed result without corrupting the output path.

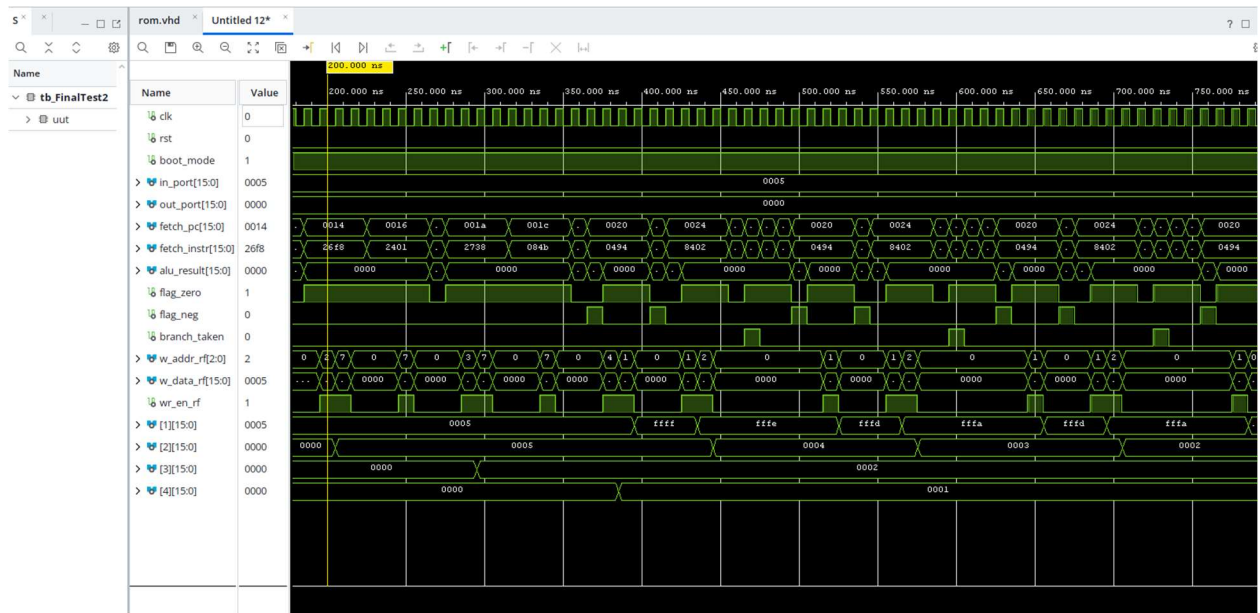


Figure 16: Final Test 2 Waveform showing looping behavior

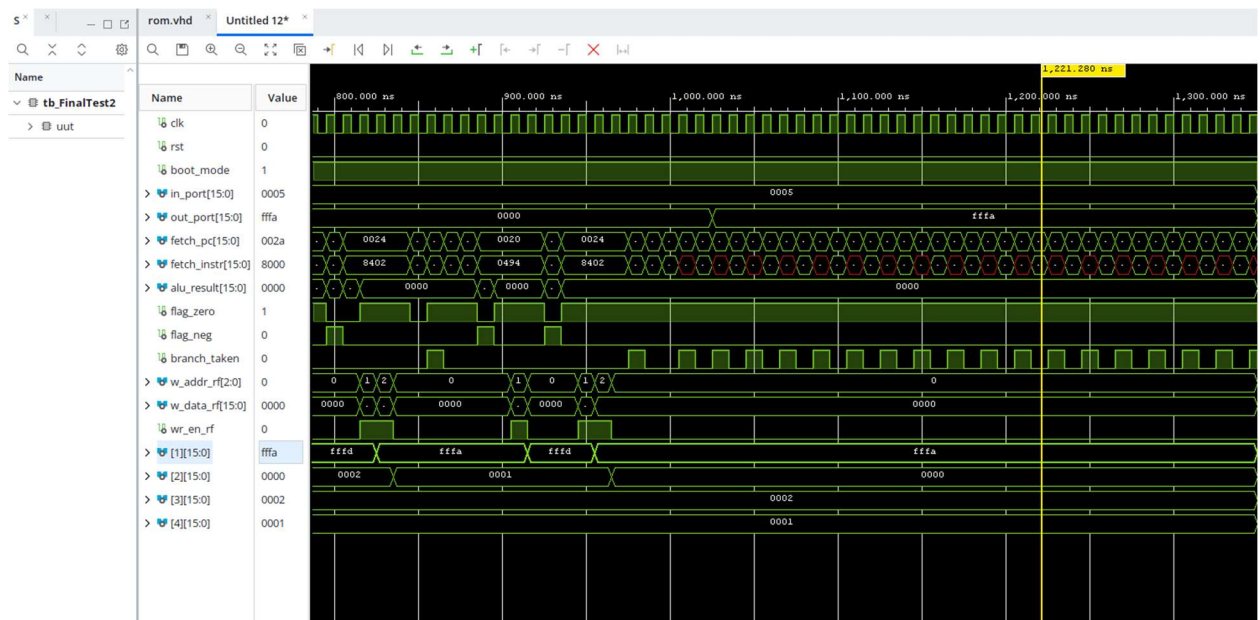


Figure 17: Waveform showing final expected outputs

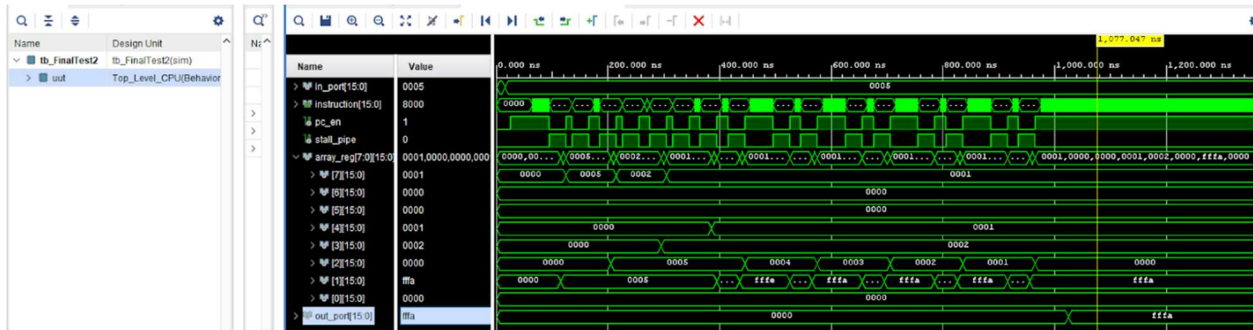


Figure 18: Final Test 2 Waveform

Factorial Test Waveform

The basic factorial test uses input 0x0005 and should produce 0x0078, which is decimal 120, on the output port. The program initializes the decrement constant and accumulator, multiplies downward through the loop, subtracts the counter each cycle, and exits once the TEST/BRR.N path determines the loop is complete. This test validates repeated multiply-subtract sequencing, negative-flag generation for loop termination, and stable output behavior for a non-overflowing factorial computation.

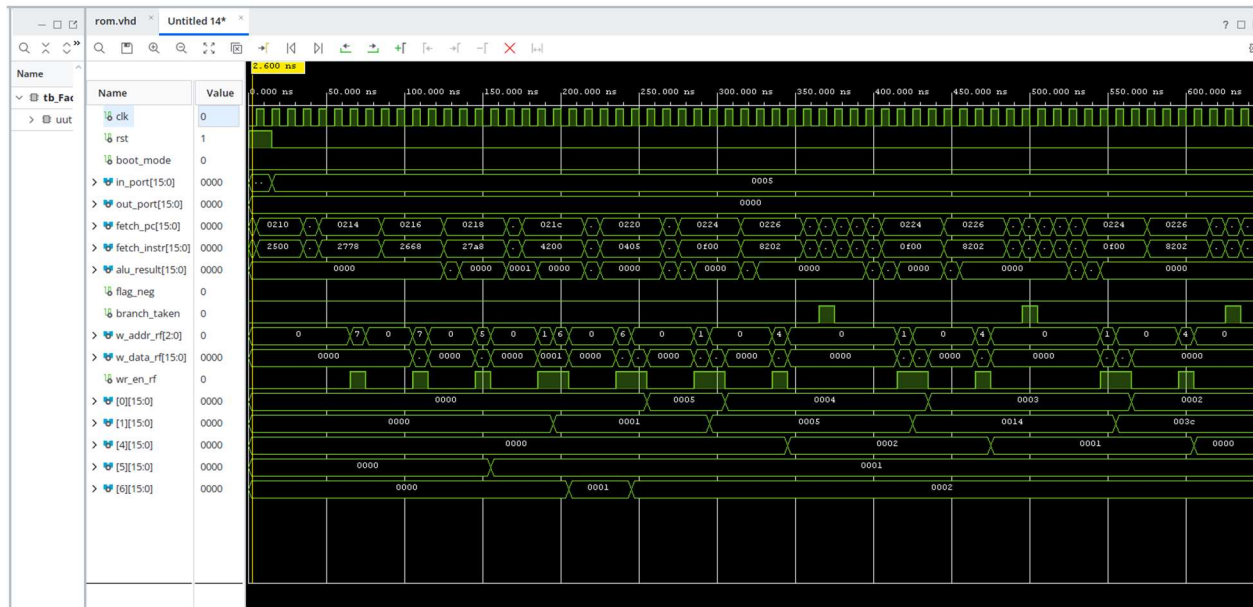


Figure 19: Factorial Test Waveform, 0ns-600ns

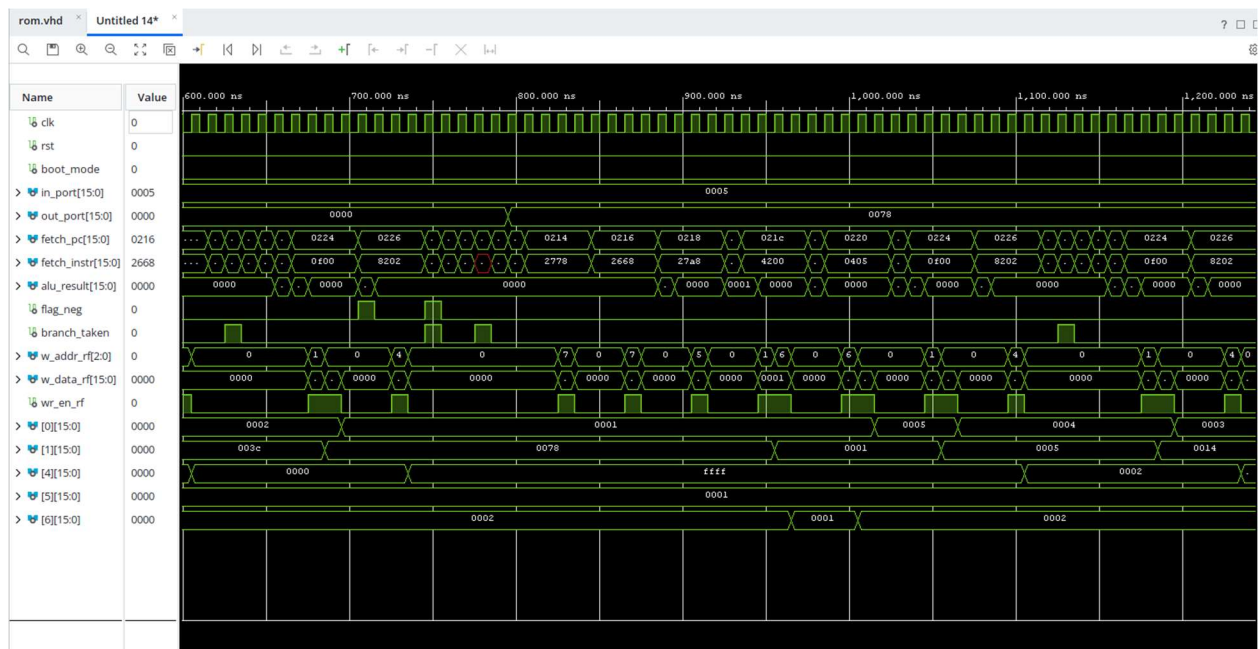


Figure 20: Factorial Test Waveform, 600ns - 1200ns

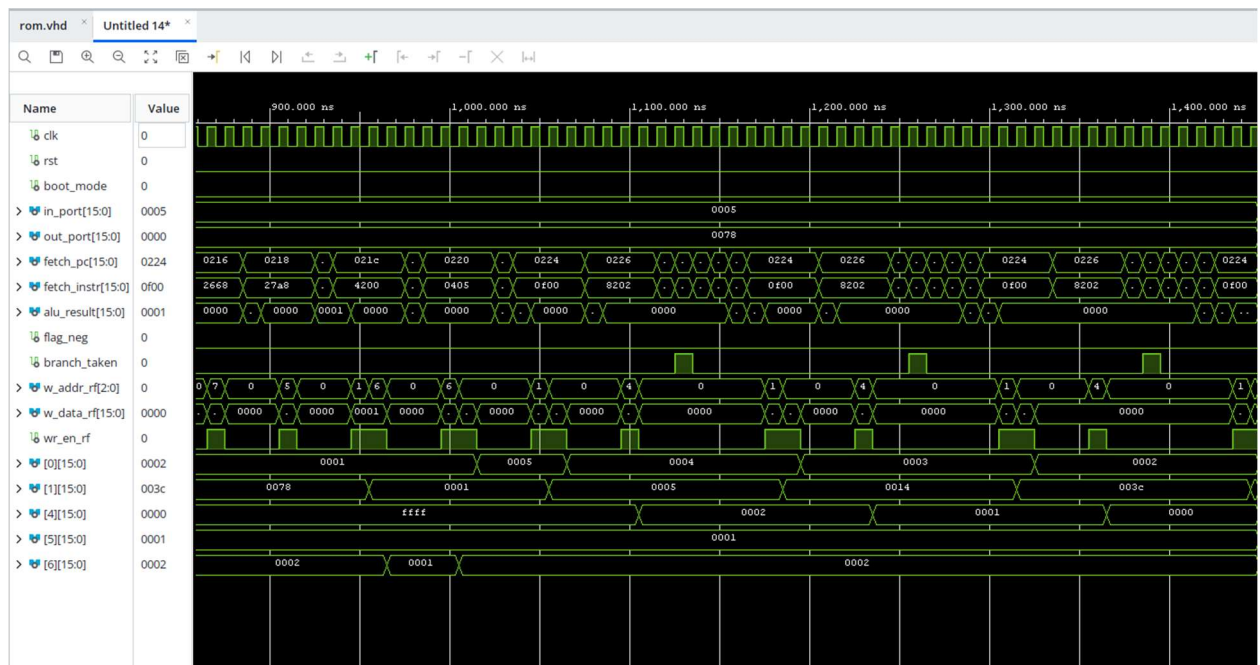


Figure 21: Factorial Test Waveform, final expected outputs

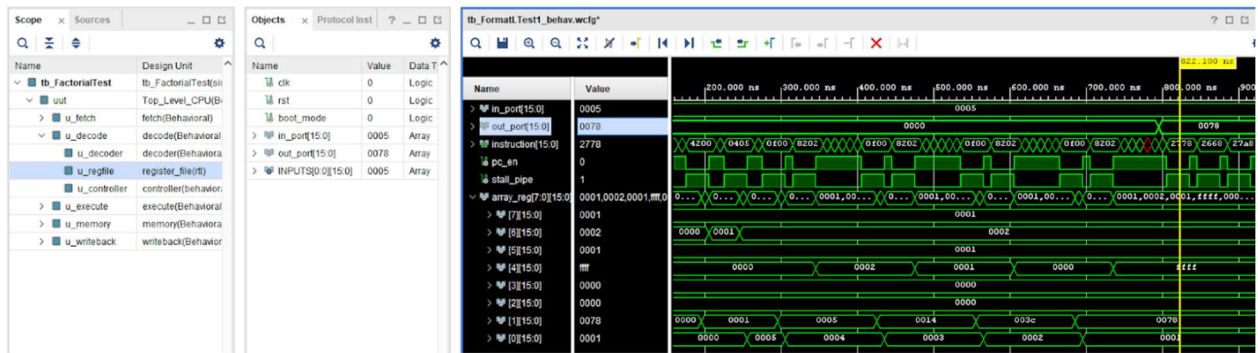


Figure 22: Factorial Test Waveform

Factorial Test Overflow Waveform:

The overflow factorial test uses input 0x0008, because $8! = 40320$ exceeds the signed 16-bit range even though it still fits in unsigned 16-bit form. On the overflowing multiply, the full product is 0x00009D80, the ALU truncates the stored 16-bit result to 0x9D80, and the new overflow flag causes BRR.overflow to jump to the overflow handler, which outputs 0x0000. This test demonstrates the hardware/software co-design path for overflow detection by combining a 32-bit multiply check in the ALU with a dedicated overflow-sensitive branch in control flow.

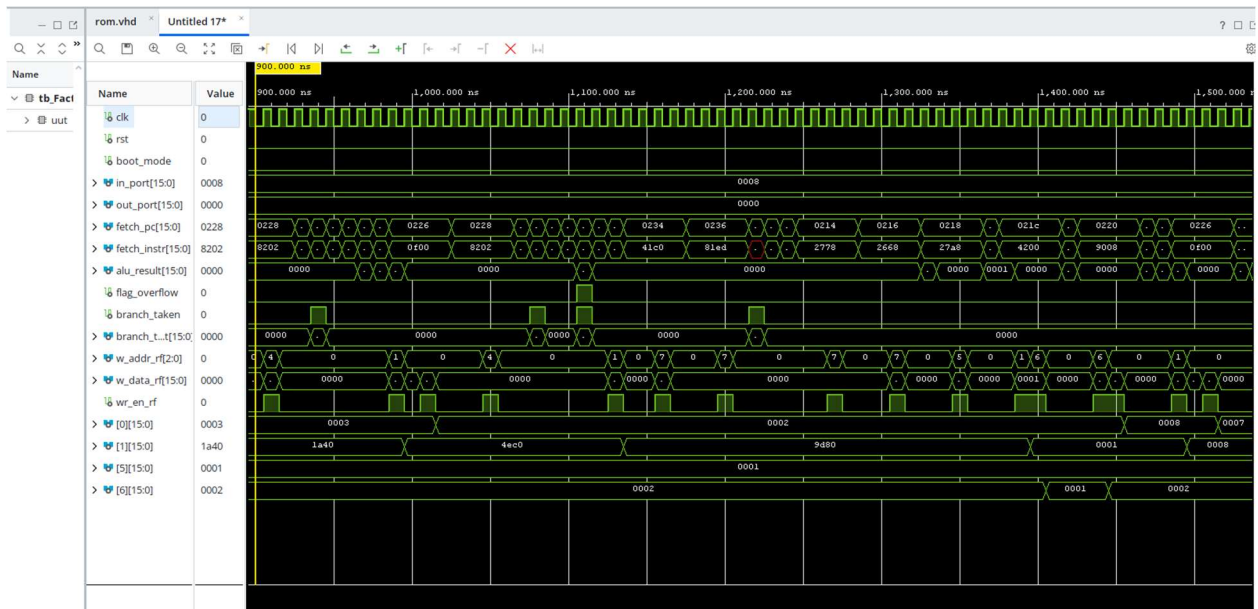


Figure 23: Factorial Test Waveform 900ns - 1500ns showing Factorial Behavior and then resetting as per looping behavior

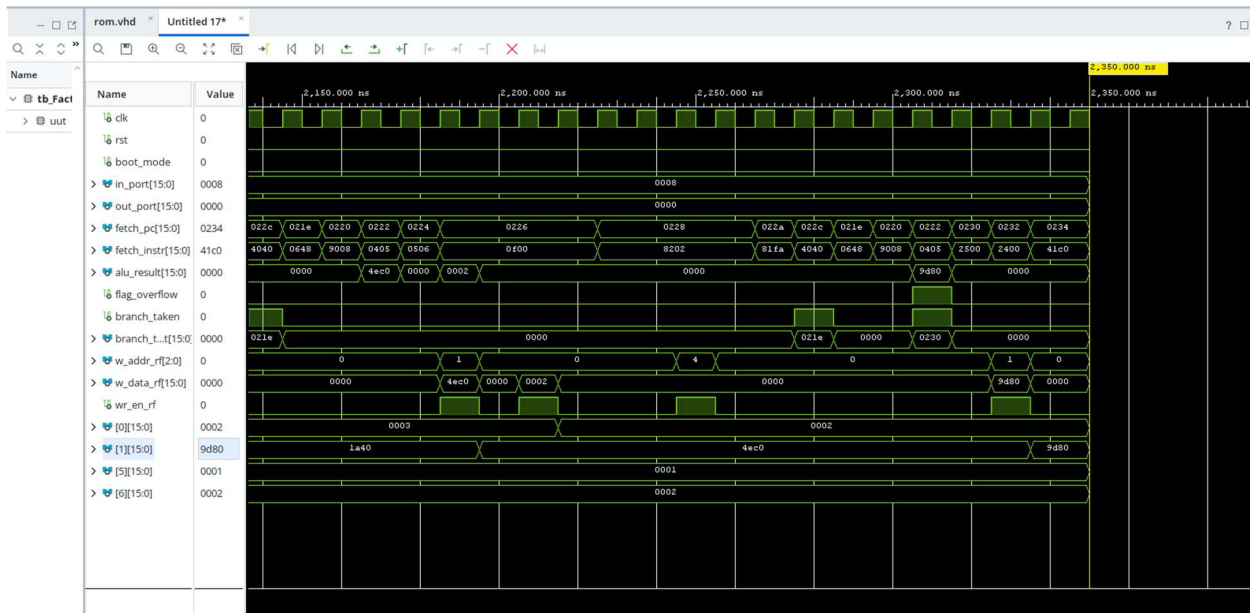


Figure 24: Factorial Test Waveform with expected outputs

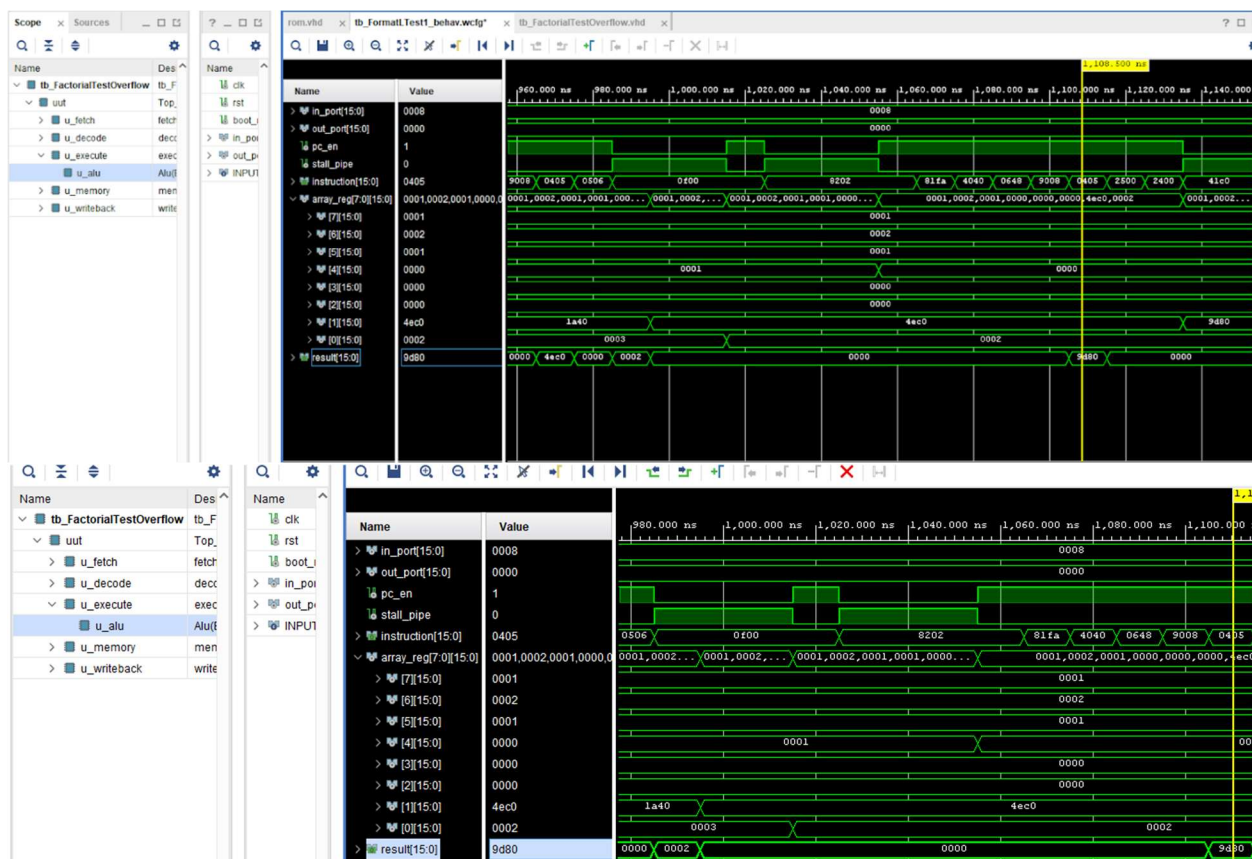


Figure 25: Factorial Overflow Test Waveforms

4. Conclusion

Across the complete verification pass, the captured waveforms matched the intended architectural behavior and satisfied the criteria described in the Format B, Format L, final-demo, and factorial rubrics. For Format B, the waveforms confirmed correct input loading, RAW hazard handling, branch and return control flow, loop behavior, signed arithmetic, and the expected final register states; for Format L, they confirmed correct LOADIMM and MOV initialization followed by a successful STORE and LOAD round-trip through memory. For the final tests and factorial demonstrations, the waveforms showed the required iterative behavior, branch-driven loop exits, correct final output values, and, in the overflow case, correct assertion of the overflow flag with the fail-safe output path to 0x0000.

Format A had already been demonstrated earlier in the project, but because the design grew significantly after that point, we re-ran it as a sanity check to ensure the newer pipeline, branch, memory, and overflow changes had not regressed the simpler arithmetic and I/O path. That sanity check matched expectations as well, with the output reconfirmed at 0x0060 and the intermediate behavior remaining consistent with the original intent of the test. Overall, the waveform evidence supports the conclusion that the final integrated CPU implementation behaves as expected for the tested instruction formats, control-flow cases, I/O behavior, memory operations, and overflow handling requirements.

—